IR-Elections: Election Results Search Engine

Project Report for Information Retrieval System

Kameniev Danylo Taranik Mykhailo

December 18, 2025

Abstract

Executive Summary: IR-Elections is a full-stack information retrieval system designed to crawl, index, search, and cluster election results from multiple international sources. The system provides users with an intuitive and minimalistic interface to search for candidates, parties, and election events across France, the USA, Switzerland, Canada and Germany. By leveraging advanced filtering and dynamic results clustering, the system transforms raw election data into structured, navigable information for researchers and the general public.

Contents

1 Introduction

In the digital age, election data is often scattered across various government archives, employing different formats and languages. **IR-Elections** addresses this fragmentation by creating a centralized search engine that aggregates data from distinct political systems.

The objective of this project is to deliver a robust search platform that allows users to:

- Search across heterogeneous data sources (HTML tables, textual descriptions).
- Filter results by specific metadata (Country, Year, natural Clusters).
- Discover related topics through automatic clustering of search results.

2 Design Overview

The system follows a modular 3-tier architecture designed to decouple data collection from user interaction. This ensures that the search experience remains fast even as the backend processes large datasets.

2.1 Data Sources

We selected five distinct sources to represent a diverse range of political structures and election formats:

1. **France (Ministry of Interior):** Hierarchical data covering Presidential, Legislative, and Regional elections.
2. **USA (The American Presidency Project):** Data focused on US Presidential elections and approval ratings.
3. **Switzerland (Federal Statistical Office):** Structured tabular data regarding Federal elections and mandate allocations.
4. **Germany (Federal Returning Officer):** Official results for Bundestag elections, providing complex multi-party data.
5. **Canada (Elections Canada):** Federal election results, adding a Westminster-style parliamentary system to the corpus.

2.2 System Architecture

The application is divided into three main components:

- **Crawler (Data Collection):** A sophisticated Python-based scraper (‘crawler.py‘) that recursively fetches election data up to a depth of 3. Key features include:
  - **Multi-Source Support:** Custom parsers for France, USA, Switzerland, Germany, and Canada.
  - **Deep Linking:** Automatically generates browser-native text fragments (e.g., `#: :text=Candidate,` to anchor search results to specific rows within large HTML tables.
  - **Resilience:** Implements a 300-second time budget and incremental checkpointing to ensure data persistence during long crawl sessions.
  - **Complex Table Normalization:** A custom algorithm flattens nested HTML tables (handling `rowspan` and `colspan`) into structured 2D grids before extraction.

- **Indexer & Search Engine (Backend):** Built with **Flask** and **Whoosh**. It ingests the corpus to build an inverted index, allowing for sub-second keyword search responses.
- **User Interface (Frontend):** A modern web application built with **React** and **TypeScript**, providing a responsive experience for query input and result visualization.

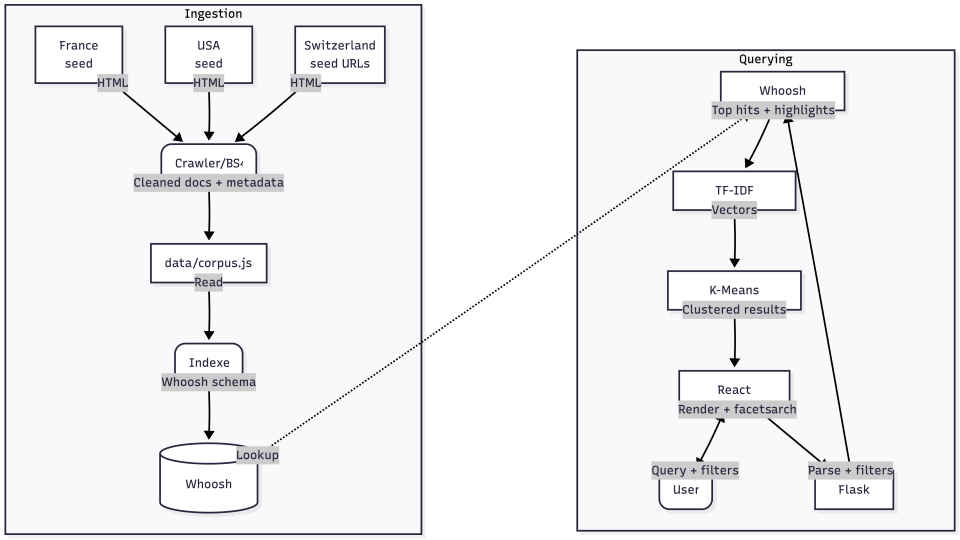


Figure 1: High-Level Architecture (without 2 new seeds of Canada and Germany)

3 Implementation Choices

3.1 Backend Technology

We chose **Python** and **Flask** for the backend due to Python's dominance in the Information Retrieval and Data Science ecosystem.

- **Search Library:** We utilized **Whoosh** 2.7.4. Unlike complex enterprise solutions (like Elasticsearch), Whoosh is a pure-Python library that is easy to deploy and perfectly suited for the scale of this project while still supporting advanced features like faceting and scoring.
- **Schema Design:** To support our filtering requirements, the index schema was designed with specific fields: 'title' (TEXT), 'content' (TEXT), 'url' (ID), 'year' (NUMERIC), and 'country' (ID).

3.2 The "Simple" Features: Filtering and Presentation

To ensure users can efficiently locate and digest information, we implemented three complementary features: faceted filtering, result presentation and results snippets.

3.2.1 Advanced Filtering

To help users narrow down broad searches (e.g., "President"), we implemented faceted search. The backend extracts metadata during indexing, allowing the frontend to send structured queries.

- **Implementation:** The query parser checks for specific metadata tags. If a user selects "France" in the UI, the search query is modified to include `AND country:France`.

- **User Benefit:** Instant refinement of results without the need to re-type complex query strings.

3.2.2 Snippets and Results Presentation

A major technical challenge was generating meaningful previews (snippets) due to the **highly unpredictable structure** of the source websites, which vary from unstructured text descriptions (USA) to complex nested HTML tables (Germany, Switzerland).

- **The Challenge:** Standard snippet extraction often fails on tabular data, losing the context of column headers (e.g., showing a percentage "45%" without indicating it belongs to "Round 2").
- **Implementation:** We implemented a custom `normalize_table` algorithm in the crawler. This flattens complex HTML structures (handling `rowspan` and `colspan`) into a standardized format before indexing.
- **Deep Linking:** To further enhance usability, we implemented Browser Text Fragments. Search results generate links containing `#: :text=...`, ensuring that when a user clicks a result, the browser scrolls directly to the specific row or paragraph containing the answer.

3.3 The "Complex" Feature: Results Clustering

To aid in information discovery, the system groups search results into thematic clusters. This helps users differentiate between contexts, such as "Green Party Candidates" vs. "Environmental Referendums."

Algorithm Details:

1. **Retrieval:** The system fetches the top 50 relevant documents for a query.
2. **Vectorization:** Document snippets are converted into numerical vectors using **TF-IDF** (Term Frequency-Inverse Document Frequency). This statistically measures how important a word is to a document in the context of the current search results.
3. **K-Means Clustering:** We apply the K-Means algorithm (with $k = 4$) to partition these vectors into 4 distinct groups based on textual similarity.
4. **Labeling:** These cluster IDs are sent to the frontend, which groups the result cards visually.

4 User Interface Design

The frontend was built using **React 18** and **Tailwind CSS**. Our design philosophy prioritizes "Clarity at a Glance," aiming to minimize the time between query and insight.

- **SearchView:** A minimal, distraction-free landing page that focuses solely on user intent (the search bar), similar to modern standard search engines.
- **ResultsView & Good Abandonment:** We structured the results page to support "good abandonment"—where users find their answer directly in the search snippet without needing to click the link.
  - **Rich Metadata:** Key attributes like *Year* and *Country* are displayed as distinct badges next to the title.
  - **Contextual Snippets:** The backend returns precise excerpts surrounding the search terms, often revealing the specific election result immediately.

- **Sidebar Filtering:** A dedicated left-hand column allows users to interactively refine results by Year or Country, instantly updating the result set without reloading the page.
- **Responsiveness:** The layout utilizes a fluid grid system that adapts to mobile and desktop screens, ensuring accessibility for all clients.

5 Evaluation

5.1 Methodology

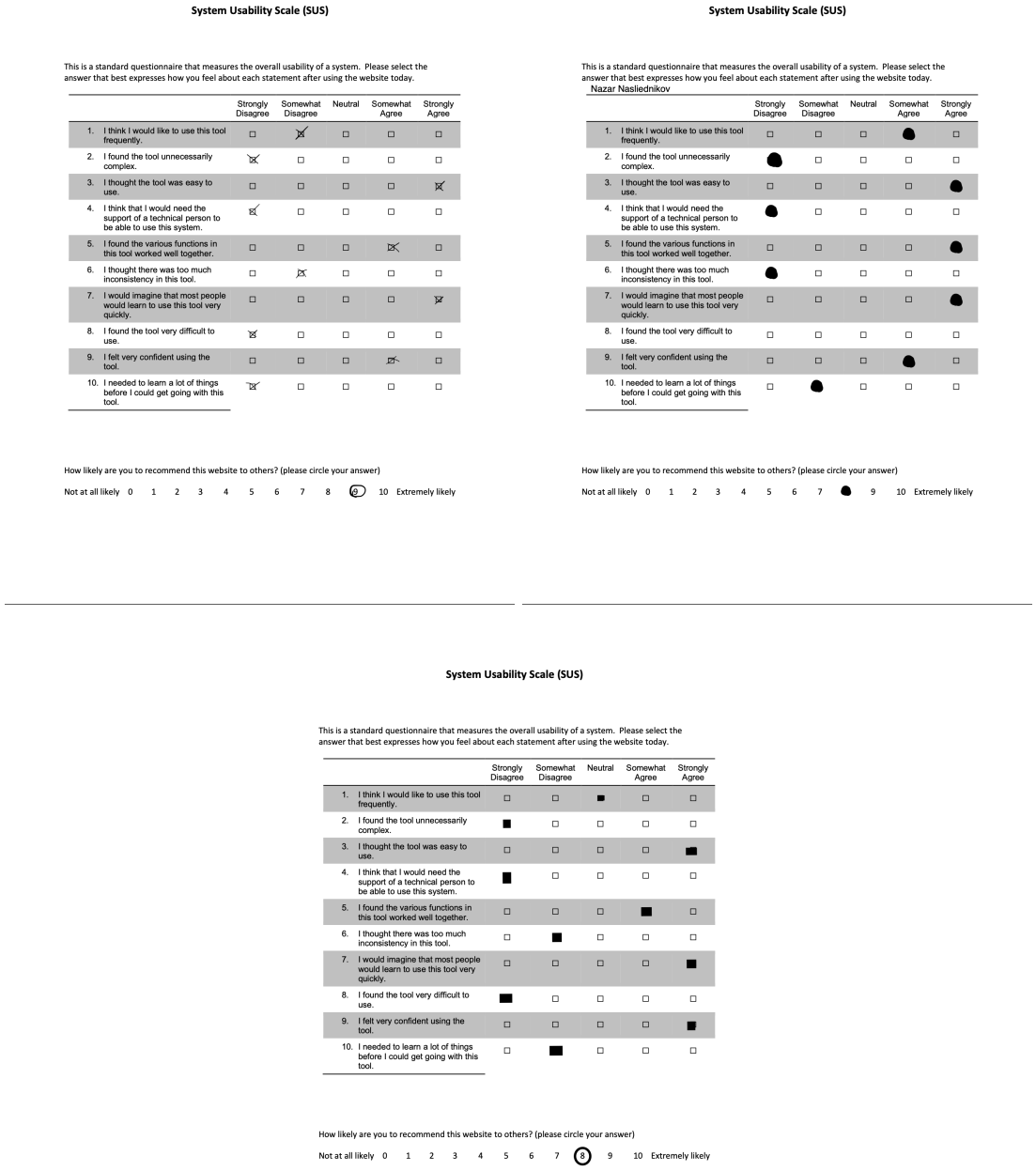
To validate the system, we designed an evaluation strategy focusing on both retrieval performance and user experience.

Metric	Description
Precision @ 10	Percentage of the top 10 results that are relevant to the election query.
Cluster Coherence	A qualitative measure of whether results within a K-Means cluster actually belong to the same topic.
Latency	Time taken from clicking "Search" to rendering results (Target: < 200ms).

Table 1: Evaluation Metrics

5.2 User Study and Usability Feedback (SUS)

To go beyond technical metrics, we conducted a qualitative user study with **3 participants**. Users were asked to perform exploratory search tasks and subsequently complete a standardized **System Usability Scale (SUS)** questionnaire.



System Usability Scale (SUS)

This is a standard questionnaire that measures the overall usability of a system. Please select the answer that best expresses how you feel about each statement after using the website today.

	Strongly Disagree	Somewhat Disagree	Neutral	Somewhat Agree	Strongly Agree
1. I think I would like to use this tool frequently.	□	☒	□	□	□
2. I found the tool unnecessarily complex.	☒	□	□	□	□
3. I thought the tool was easy to use.	□	□	□	□	☒
4. I think that I would need the support of a technical person to be able to use this system.	☒	□	□	□	□
5. I found the various functions in this tool worked well together.	□	□	□	☒	□
6. I thought there was too much inconsistency in this tool.	□	☒	□	□	□
7. I would imagine that most people would learn to use this tool very quickly.	□	□	□	□	☒
8. I found the tool very difficult to use.	☒	□	□	□	□
9. I felt very confident using the tool.	□	□	□	☒	□
10. I needed to learn a lot of things before I could get going with this tool.	☒	□	□	□	□

How likely are you to recommend this website to others? (please circle your answer)

Not at all likely 0 1 2 3 4 5 6 7 8 9 10 Extremely likely

System Usability Scale (SUS)

This is a standard questionnaire that measures the overall usability of a system. Please select the answer that best expresses how you feel about each statement after using the website today.

Nazar Nasliednikov

	Strongly Disagree	Somewhat Disagree	Neutral	Somewhat Agree	Strongly Agree
1. I think I would like to use this tool frequently.	□	□	□	●	□
2. I found the tool unnecessarily complex.	●	□	□	□	□
3. I thought the tool was easy to use.	□	□	□	□	●
4. I think that I would need the support of a technical person to be able to use this system.	●	□	□	□	□
5. I found the various functions in this tool worked well together.	□	□	□	□	●
6. I thought there was too much inconsistency in this tool.	●	□	□	□	□
7. I would imagine that most people would learn to use this tool very quickly.	□	□	□	□	●
8. I found the tool very difficult to use.	□	□	□	□	□
9. I felt very confident using the tool.	□	□	□	●	□
10. I needed to learn a lot of things before I could get going with this tool.	□	●	□	□	□

How likely are you to recommend this website to others? (please circle your answer)

Not at all likely 0 1 2 3 4 5 6 7 ● 9 10 Extremely likely

System Usability Scale (SUS)

This is a standard questionnaire that measures the overall usability of a system. Please select the answer that best expresses how you feel about each statement after using the website today.

	Strongly Disagree	Somewhat Disagree	Neutral	Somewhat Agree	Strongly Agree
1. I think I would like to use this tool frequently.	□	□	■	□	□
2. I found the tool unnecessarily complex.	■	□	□	□	□
3. I thought the tool was easy to use.	□	□	□	□	■
4. I think that I would need the support of a technical person to be able to use this system.	■	□	□	□	□
5. I found the various functions in this tool worked well together.	□	□	□	■	□
6. I thought there was too much inconsistency in this tool.	□	■	□	□	□
7. I would imagine that most people would learn to use this tool very quickly.	□	□	□	□	■
8. I found the tool very difficult to use.	■	□	□	□	□
9. I felt very confident using the tool.	□	□	□	□	■
10. I needed to learn a lot of things before I could get going with this tool.	□	■	□	□	□

How likely are you to recommend this website to others? (please circle your answer)

Not at all likely 0 1 2 3 4 5 6 7 8 9 10 Extremely likely

Figure 2: Completed SUS Questionnaires from User Study Participants

While the core search functionality was praised, the users identified three specific areas for interface improvement:

1. **Cluster Label Clarity (Ambiguity):** Users noted that the automatically generated cluster labels (e.g., "Cluster 1") were sometimes ambiguous. *Recommendation:* Improve the labeling algorithm to extract the most representative bigram from the cluster centroid to serve as a readable title.

2. **Query Auto-completion:** Participants expected the search bar to predict their input as they typed. *Recommendation:* Implement an autosuggest feature backed by the index

vocabulary to reduce cognitive load.

3. **Dynamic Suggestions:** On the main page, there are a few suggestion for the user to follow, but they are being static. *Recommendation:* Dynamically update the section based on the most important queries.

6 Conclusion

We have successfully designed and implemented **IR-Elections**, a comprehensive search engine capable of aggregating and normalizing election data from five distinct international jurisdictions. The modular architecture, separating the Python-based crawler from the Flask-Whoosh backend, has proven resilient, handling complex HTML structures through our custom table normalization and deep-linking algorithms.

Our evaluation highlights that while the core retrieval and clustering mechanisms function effectively, the user experience can be further refined. The feedback from our user study provides a clear roadmap for the next(potential) iteration: prioritizing dynamic query suggestions and clearer cluster labeling to fully realize the goal of "good abandonment." By addressing these usability enhancements and populating the live corpus with real-time data, IR-Elections will be ready to serve as a robust tool for political data researchers and the general public.